P0847R2
Deducing this

Published Proposal, 2019-01-15

This version:
http://wg21.link/P0847

Authors:
Gašper Ažman (gasper dot azman at gmail dot com)
Simon Brand (simon dot brand at microsoft dot com)
Ben Deane (ben at elbeno dot com)
Barry Revzin (barry dot revzin at gmail dot com)

Audience:
EWG, SG7

Project:
ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Abstract

We propose a new mechanism for specifying or deducing the value category of an instance of a class — in other words, a way to tell from within a member function whether the object it's invoked on is an lvalue or an rvalue; whether it is const or volatile; and the object's type.

Table of Contents

§ 1. Revision History

§ 1.1. Changes since r1

[P0847R1] was presented in San Diego in November 2018 with a wide array of syntaxes and name lookup options. Discussion there revealed some potential issues with regards to lambdas that needed to be ironed out. This revision zeroes in on one specific syntax and name lookup semantic which solves all the use-cases.

§ 1.2. Changes since r0

[P0847R0] was presented in Rapperswil in June 2018 using a syntax adjusted from the one used in that paper, using `this Self&& self` to indicate the explicit object parameter rather than the `Self&& this self` that appeared in r0 of our paper.

EWG strongly encouraged us to look in two new directions:

- a different syntax, placing the object parameter's type after the member function's parameter declarations (where the *cv-ref* qualifiers are today)
- a different name lookup scheme, which could prevent implicit/unqualified access from within new-style member functions that have an explicit self-type annotation, regardless of syntax.

This revision carefully explores both of these directions, presents different syntaxes and lookup schemes, and discusses in depth multiple use cases and how each syntax can or cannot address them.

§ 2. Motivation

In C++03, member functions could have *cv*-qualifications, so it was possible to have scenarios where a particular class would want both a `const` and non-`const` overload of a particular member. (Note that it was also possible to want `volatile` overloads, but those are less common and thus are not examined here.) In these cases, both overloads do the same thing — the only difference is in the types being accessed and used. This was handled by either duplicating the function while adjusting types and qualifications as necessary, or having one overload delegate to the other. An example of the latter can be found in Scott Meyers's "Effective C++" [Effective], Item 3:

```
class TextBlock {
public:
  char const& operator[](size_t position) const {
    // ...
    return text[position];
  }

  char& operator[](size_t position) {
    return const_cast<char&>(
      static_cast<TextBlock const&>(*this)[position]
    );
  }
  // ...
};
```

Arguably, neither duplication nor delegation via `const_cast` are great solutions, but they work.

In C++11, member functions acquired a new axis to specialize on: ref-qualifiers. Now, instead of potentially needing two overloads of a single member function, we might need four: `&`, `const&`, `&&`, or `const&&`. We have three approaches to deal with this:

- We implement the same member four times;
- We have three overloads delegate to the fourth; or
- We have all four overloads delegate to a helper in the form of a private static member function.

One example of the latter might be the overload set for `optional<T>::value()`, implemented as:

Quadruplication

```
template <typename T>
class optional {
  // ...
  constexpr T& value() & {
    if (has_value()) {
      return this->m_value;
    }
    throw bad_optional_access();
  }

  constexpr T const& value() const& {
    if (has_value()) {
      return this->m_value;
    }
    throw bad_optional_access();
  }

  constexpr T&& value() && {
    if (has_value()) {
      return move(this->m_value);
    }
    throw bad_optional_access();
  }

  constexpr T const&&
  value() const&& {
    if (has_value()) {
      return move(this->m_value);
    }
    throw bad_optional_access();
  }
  // ...
};
```

Delegation to 4th

```
template <typename T>
class optional {
  // ...
  constexpr T& value() & {
    return const_cast<T&>(
      static_cast<optional const&>(
        *this).value());
  }

  constexpr T const& value() const& {
    if (has_value()) {
      return this->m_value;
    }
    throw bad_optional_access();
  }

  constexpr T&& value() && {
    return const_cast<T&&>(
      static_cast<optional const&>(
        *this).value());
  }

  constexpr T const&&
  value() const&& {
    return static_cast<T const&&>(
      value());
  }
  // ...
};
```

Delegation t

```
template <typename T
class optional {
  // ...
  constexpr T& value
    return value_imp
  }

  constexpr T const&
    return value_imp
  }

  constexpr T&& valu
    return value_imp
  }

  constexpr T const&
  value() const&& {
    return value_imp
  }

private:
  template <typename
  static decltype(au
  value_impl(Opt&& o
    if (!opt.has_val
      throw bad_opti
    }
    return forward<O
  }
  // ...
};
```

This is far from a complicated function, but essentially repeating the same code four times — or using artificial delegation to avoid doing so — begs a rewrite. Unfortunately, it's impossible to improve; we *must* implement it this way. It seems we should be able to abstract away the qualifiers as we can for non-member functions, where we simply don't have this problem:

```
template <typename T>
class optional {
    // ...
    template <typename Opt>
    friend decltype(auto) value(Opt&& o) {
        if (o.has_value()) {
            return forward<Opt>(o).m_value;
        }
        throw bad_optional_access();
    }
    // ...
};
```

All four cases are now handled with just one function... except it's a non-member function, not a member function. Different semantics, different syntax, doesn't help.

There are many cases where we need two or four overloads of the same member function for different `const`- or ref-qualifiers. More than that, there are likely additional cases where a class should have four overloads of a particular member function but, due to developer laziness, doesn't. We think that there are enough such cases to merit a better solution than simply "write it, write it again, then write it two more times."

§ 3. Proposal

We propose a new way of declaring non-static member functions that will allow for deducing the type and value category of the class instance parameter while still being invocable with regular member function syntax. This is a strict extension to the language.

We believe that the ability to write *cv-ref qualifier*-aware member function templates without duplication will improve code maintainability, decrease the likelihood of bugs, and make fast, correct code easier to write.

The proposal is sufficiently general and orthogonal to allow for several new exciting features and design patterns for C++:

- recursive lambdas
- a new approach to mixins, a CRTP without the CRT
- move-or-copy-into-parameter support for member functions
- efficiency by avoiding double indirection with invocation
- perfect, sfinae-friendly call wrappers

These are explored in detail in the examples section.

This proposal assumes the existence of two library additions, though it does not propose them:

- `like_t`, a metafunction that applies the *cv*- and *ref*-qualifiers of the first type onto the second (e.g. `like_t<int&, double>` is `double&`, `like_t<X const&&, Y>` is `Y const&&`, etc.)
- `forward_like`, a version of `forward` that is intended to forward a variable not based on its own type but instead based on some other type. `forward_like<T>(u)` is short-hand for `forward<like_t<T,decltype(u)>>(u)`.

§ 3.1. Proposed Syntax

The proposed syntax in this paper is to use an explicit `this`-annotated parameter.

A non-static member function can be declared to take as its first parameter an *explicit object parameter*, denoted with the prefixed keyword `this`. Once we elevate the object parameter to a proper function parameter, it can be deduced following normal function template deduction rules:

```
struct X {
    void foo(this X const& self, int i);

    template <typename Self>
    void bar(this Self&& self);
};

struct D : X { };

void ex(X& x, D const& d) {
    x.foo(42);      // 'self' is bound to 'x', 'i' is 42
    x.bar();        // deduces Self as X&, calls X::bar<X&>
    move(x).bar();  // deduces Self as X, calls X::bar<X>

    d.foo(17);      // 'self' is bound to 'd'
    d.bar();        // deduces Self as D const&, calls X::bar<D const&>
}
```

Member functions with an explicit object parameter cannot be `static` or have *cv*- or *ref*-qualifiers.

A call to a member function will interpret the object argument as the first (`this`-annotated) parameter to it; the first argument in the parenthesized expression list is then interpreted as the second parameter, and so forth.

Following normal deduction rules, the template parameter corresponding to the explicit object parameter can deduce to a type derived from the class in which the member function is declared, as in the example above for `d.bar()`).

We can use this syntax to implement `optional::value()` and `optional::operator->()` in just two functions instead of the current six:

```
template <typename T>
struct optional {
  template <typename Self>
  constexpr auto&& value(this Self&& self) {
    if (!self.has_value()) {
      throw bad_optional_access();
    }

    return forward<Self>(self).m_value;
  }

  template <typename Self>
  constexpr auto operator->(this Self&& self) {
    return addressof(self.m_value);
  }
};
```

This syntax can be used in lambdas as well, with the `this`-annotated parameter exposing a way to refer to the lambda itself in its body:

```
vector captured = {1, 2, 3, 4};
[captured](this auto&& self) -> decltype(auto) {
  return forward_like<decltype(self)>(captured);
}

[captured]<class Self>(this Self&& self) -> decltype(auto) {
  return forward_like<Self>(captured);
}
```

The lambdas can either move or copy from the capture, depending on whether the lambda is an lvalue or an rvalue.

§ 3.2. Proposed semantics

What follows is a description of how deducing `this` affects all important language constructs — name lookup, type deduction, overload resolution, and so forth.

§ 3.2.1. Name lookup: candidate functions

In C++17, name lookup includes both static and non-static member functions found by regular class lookup when invoking a named function or an operator, including the call operator, on an object of class type. Non-static member functions are treated as if there were an implicit object parameter whose type is an lvalue or rvalue reference to *cv* `X` (where the reference and *cv* qualifiers are determined based on the function's own qualifiers) which binds to the object on which the function was invoked.

For non-static member functions using an explicit object parameter, lookup will work the same way as other member functions in C++17, with one exception: rather than implicitly determining the type of the object parameter based on the *cv*- and *ref*-qualifiers of the member function, these are now explicitly determined by the provided type of the explicit object parameter. The following examples illustrate this concept.

C++17	Proposed

`struct X {`
`  // implicit object has type X&`
`  void foo() &;`
 	
`  // implicit object has type X const&`
`  void foo() const&;`
 	
`  // implicit object has type X&&`
`  void bar() &&;`
`};`
	`struct X {`
	`  // explicit object has type X&`
	`  void foo(this X&);`
	
	`  // explicit object has type X const&`
	`  void foo(this X const&);`
	
	`  // explicit object has type X&&`
	`  void bar(this X&&);`
	`};`

Name lookup on an expression like `obj.foo()` in C++17 would find both overloads of `foo` in the first column, with the non-const overload discarded should `obj` be const.

With the proposed syntax, `obj.foo()` would continue to find both overloads of `foo`, with identical behaviour to C++17.

The only change in how we look up candidate functions is in the case of an explicit object parameter, where the argument list is shifted by one. The first listed parameter is bound to the object argument, and the second listed parameter corresponds to the first argument of the call expression.

This paper does not propose any changes to overload *resolution* but merely suggests extending the candidate set to include non-static member functions and member function templates written in a new syntax. Therefore, given a call to `x.foo()`, overload resolution would still select the first `foo()` overload if `x` is not `const` and the second if it is.

The behaviors of the two columns are exactly equivalent as proposed.

The only change as far as candidates are concerned is that the proposal allows for deduction of the object parameter, which is new for the language.

§ 3.2.2. Type deduction

One of the main motivations of this proposal is to deduce the *cv*-qualifiers and value category of the class object, which requires that the explicit member object or type be deducible from the object on which the member function is invoked.

If the type of the object parameter is a template parameter, all of the usual template deduction rules apply as expected:

```
struct X {
  template <typename Self>
  void foo(this Self&&, int);
};

struct D : X { };

void ex(X& x, D& d) {
    x.foo(1);       // Self=X&
    move(x).foo(2); // Self=X
    d.foo(3);       // Self=D&
}
```

It's important to stress that deduction is able to deduce a derived type, which is extremely powerful. In the last line, regardless of syntax, `Self` deduces as `D&`. This has implications for §3.2.4 Name lookup: within member functions, and leads to a potential template deduction extension.

§ 3.2.3. By value `this`

But what if the explicit type does not have reference type? What should this mean?

```
struct less_than {
    template <typename T, typename U>
    bool operator()(this less_than, T const& lhs, U const& rhs) {
        return lhs < rhs;
    }
};

less_than{}(4, 5);
```

Clearly, the parameter specification should not lie, and the first parameter (`less_than{}`) is passed by value.

Following the proposed rules for candidate lookup, the call operator here would be a candidate, with the object parameter binding to the (empty) object and the other two parameters binding to the arguments. Having a value parameter is nothing new in the language at all — it has a clear and obvious meaning, but we've never been able to take an object parameter by value before. For cases in which this might be desirable, see §4.4 By-value member functions.

§ 3.2.4. Name lookup: within member functions

So far, we've only considered how member functions with explicit object parameters are found with name lookup and how they deduce that parameter. Now we move on to how the bodies of these functions actually behave.

Since the explicit object parameter is deduced from the object on which the function is called, this has the possible effect of deducing *derived* types. We must carefully consider how name lookup works in this context.

```
struct B {
    int i = 0;

    template <typename Self> auto&& f1(this Self&&) { return i;  }
    template <typename Self> auto&& f2(this Self&&) { return this->i; }
    template <typename Self> auto&& f3(this Self&&) { return forward_like<Self>(*this).i; }
    template <typename Self> auto&& f4(this Self&&) { return forward<Self>(*this).i; }
    template <typename Self> auto&& f5(this Self&& self) { return forward<Self>(self).i; }
};

struct D : B {
    // shadows B::i
    double i = 3.14;
};
```

The question is, what do each of these five functions do? Should any of them be ill-formed? What is the safest option?

We believe that there are three approaches to choose from:

1. If there is an explicit object parameter, `this` is inaccessible, and each access must be through `self`. There is no implicit lookup of members through `this`. This makes `f1` through `f4` ill-formed and only `f5` well-formed. However, while `B().f5()` returns a reference to `B::i`, `D().f5()` returns a reference to `D::i`, since `self` is a reference to `D`.
2. If there is an explicit object parameter, `this` is accessible and points to the base subobject. There is no implicit lookup of members; all access must be through `this` or `self` explicitly. This makes `f1` ill-formed. `f2` would be well-formed and always return a reference to `B::i`. Most importantly, `this` would be *dependent* if the explicit object parameter was deduced. `this->i` is always going to be an `int` but it could be either an `int` or an `int const` depending on whether the `B` object is const. `f3` would always be well-formed and would be the correct way to return a forwarding reference to `B::i`. `f4` would be well-formed when invoked on `B` but ill-formed if invoked on `D` because of the requested implicit downcast. As before, `f5` would be well-formed.
3. `this` is always accessible and points to the base subobject; we allow implicit lookup as in C++17. This is mostly the same as the previous choice, except that now `f1` is well-formed and exactly equivalent to `f2`.

Following discussion in San Diego, the option we are proposing is #1. This allows for the clearest model of what a `this`-annotated function is: it is a `static` member function that offers a more convenient function call syntax. There is no implicit `this` in such functions, the only mention of `this` would be the annotation on the object parameter. All member access must be done directly through the object parameter.

The consequence of such a choice is that we will need to defend against the object parameter being deduced to a derived type. To ensure that `f5()` above is always returning a reference to `B::i`, we would need to write one of the following:

```
template <typename Self>
auto&& f5(this Self&& self) {
    // explicitly cast self to the appropriately qualified B
    // note that we have to cast self, not self.i
    return static_cast<like_t<Self, B>&&>(self).i;

    // use the explicit subobject syntax. Note that this is always
    // an lvalue reference - not a forwarding reference
    return self.B::i;

    // use the explicit subobject syntax to get a forwarding reference
    return forward<Self>(self).B::i;
}
```

§ 3.2.5. Writing the function pointer types for such functions

As described in the previous section, the model for a member function with an explicit object parameter is a `static` member function.

In other words, given:

```
struct Y {
    int f(int, int) const&;
    int g(this Y const&, int, int);
};
```

While the type of `&Y::f` is `int(Y::*)(int, int) const&`, the type of `&Y::g` is `int(*)(Y const&, int, int)`. As these are *just* function pointers, the usage of these two member functions differs once we drop them to pointers:

```
Y y;
y.f(1, 2); // ok as usual
y.g(3, 4); // ok, this paper

auto pf = &Y::f;
pf(y, 1, 2);              // error: pointers to member functions are not callable
(y.*pf)(1, 2);            // okay, same as above
std::invoke(pf, y, 1, 2); // ok

auto pg = &Y::g;
pg(y, 3, 4);              // okay, same as above
(y.*pg)(3, 4);            // error: pg is not a pointer to member function
std::invoke(pg, y, 3, 4); // ok
```

The rules are the same when deduction kicks in:

```
struct B {
    template <typename Self>
    void foo(this Self&&);
};

struct D : B { };
```

The type of `&B::foo<B>` is `void(*)(B&&)` and the type of `&B::foo<B const&>` is `void(*)(B const&)`. The type of `&D::foo<B>` is also `void(*)(B&&)`. This is effectively the same thing that would happen if `foo` were a normal C++17 member function. The type of `&B::foo<D>` is `void(*)(D&&)`.

By-value object parameters give you pointers to function in just the same way, the only difference being that the first parameter being a value parameter instead of a reference parameter:

```
template <typename T>
struct less_than {
    bool operator()(this less_than, T const&, T const&);
};
```

The type of `&less_than<int>::operator()` is `bool(*)(less_than<int>, int const&, int const&)` and follows the usual rules of invocation:

```
less_than<int> lt;
auto p = &less_than<int>::operator();

lt(1, 2);            // ok
p(lt, 1, 2);         // ok
(lt.*p)(1, 2);       // error: p is not a pointer to member function
invoke(p, lt, 1, 2); // ok
```

§ 3.2.6. Pathological cases

It is important to mention the pathological cases. First, what happens if `D` is incomplete but becomes valid later?

```
struct D;
struct B {
    void foo(this D&);
};
struct D : B { };
```

Following the precedent of [P0929R2], we think this should be fine, albeit strange. If `D` is incomplete, we simply postpone checking until the point of call or formation of pointer to member, etc. At that point, the call will either not be viable or the formation of pointer-to-member would be ill-formed.

For unrelated complete classes or non-classes:

```
struct A { };
struct B {
    void foo(this A&);
    void bar(this int);
};
```

The declaration can be immediately diagnosed as ill-formed.

Another interesting case, courtesy of Jens Maurer:

```
struct D;
struct B {
  int f1(this D);
};
struct D1 : B { };
struct D2 : B { };
struct D : D1, D2 { };

int x = D().f1();  // error: ambiguous lookup
int y = B().f1();  // error: B is not implicitly convertible to D
auto z = &B::f1;   // ok
z(D());            // ok
```

Even though both `D().f1()` and `B().f1()` are ill-formed, for entirely different reasons, taking a pointer to `&B::f1` is acceptable — its type is `int(*)(D)` — and that function pointer can be invoked with a `D`. Actually invoking this function does not require any further name lookup or conversion because by-value member functions do not have an implicit object parameter in this syntax (see §3.2.3 By value this).

§ 3.2.7. Teachability Implications

Explicitly naming the object as the `this`-designated first parameter fits within many programmers' mental models of the `this` pointer being the first parameter to member functions "under the hood" and is comparable to its usage in other languages, e.g. Python and Rust. It also works as a more obvious way to teach how `std::bind`, `std::thread`, `std::function`, and others work with a member function pointer by making the pointer explicit.

As such, we do not believe there to be any teachability problems.

§ 3.2.8. Can `static` member functions have an explicit object type?

No. Static member functions currently do not have an implicit object parameter, and therefore have no reason to provide an explicit one.

§ 3.2.9. Interplays with capturing `[this]` and `[*this]` in lambdas

Interoperability is perfect, since they do not impact the meaning of `this` in a function body. The introduced identifier `self` can then be used to refer to the lambda instance from the body.

3.2.10. Parsing issues

The proposed syntax has no parsings issue that we are aware of.

3.2.11. Code issues

There are two programmatic issues with this proposal that we are aware of:

1. Inadvertently referencing a shadowing member of a derived object in a base class `this`-annotated member function. There are some use cases where we would want to do this on purposes (see §4.2 CRTP, without the C, R, or even T), but for other use-cases the programmer will have to be aware of potential issues and defend against them in a somewhat verobse way.
2. Because there is no way to _just_ deduce `const` vs non-`const`, the only way to deduce the value category would be to take a forwarding reference. This means that potentially we create four instantiations when only two would be minimally necessary to solve the problem. But deferring to a templated implementation is an acceptable option and has been improved by no longer requiring casts. We believe that the problem is minimal.

3.3. Potential Extension

This extension is not explicitly proposed proposed by our paper, since it has not yet been completely explored. Nevertheless, the authors believe that certain concerns raised by the proposed feature may be alleviated by discussing the following possible solution to those issues.

One of the pitfalls of having a deduced object parameter is when the intent is solely to deduce the *cv*-qualifiers and value category of the object parameter, but a derived type is deduced as well — any access through an object that might have a derived type could inadvertently refer to a shadowed member in the derived class. While this is desirable and very powerful in the case of mixins, it is not always desirable in other situations. Superfluous template instantiations are also unwelcome side effects.

One family of possible solutions could be summarized as **make it easy to get the base class pointer**. However, all of these solutions still require extra instantiations. For `optional::value()`, we really only want four instantiations: `&`, `const&`, `&&`, and `const&&`. If something inherits from `optional`, we don't want additional instantiations of those functions for the derived types, which won't do anything new, anyway. This is code bloat.

C++ already has this long-recognised problem for free function templates. The authors have heard many a complaint about it from library vendors, even before this paper was introduced, as it is desirable to only deduce the ref-qualifier in many contexts. Therefore, it might make sense to tackle this issue in a more general way. A complementary feature could be proposed to constrain *type deduction* as opposed to removing candidates once they are deduced (as accomplished by `requires`), with the following straw-man syntax:

```
struct Base {
    template <typename Self : Base>
    auto front(this Self&& self);
};
struct Derived : Base { };

// also works for free functions
template <typename T : Base>
void foo(T&& x) {
   static_assert(is_same_v<Base, remove_reference_t<T>>);
}

Base{}.front(); // calls Base::front<Base>
Derived{}.front(); // also calls Base::front<Base>

foo(Base{}); // calls foo<Base>
foo(Derived{}); // also calls foo<Base>
```

This would create a function template that only generates functions taking a `Base`, ensuring that we don't generate additional instantiations when those functions participate in overload resolution. Such a proposal would also change how

templates participate in overload resolution, however, and is not to be attempted haphazardly.

§ 4. Real-World Examples

What follows are several examples of the kinds of problems that can be solved using this proposal.

§ 4.1. Deduplicating Code

This proposal can de-duplicate and de-quadruplicate a large amount of code. In each case, the single function is only slightly more complex than the initial two or four, which makes for a huge win. What follows are a few examples of ways to reduce repeated code.

This particular implementation of optional is Simon's, and can be viewed on GitHub. It includes some functions proposed in [P0798R0], with minor changes to better suit this format:

C++17

```
class TextBlock {
public:
  char const& operator[](size_t position) const {
    // ...
    return text[position];
  }

  char& operator[](size_t position) {
    return const_cast<char&>(
      static_cast<TextBlock const&>
        (this)[position]
    );
  }
  // ...
};
```

Proposed

```
class TextBlock {
public:
  template <typename Self>
  auto& operator[](this Self&& self, size_t positio
    // ...
    return self.text[position];
  }
  // ...
};
```

C++17

```
template <typename T>
class optional {
  // ...
  constexpr T* operator->() {
    return addressof(this->m_value);
  }

  constexpr T const*
  operator->() const {
    return addressof(this->m_value);
  }
  // ...
};
```

Proposed

```
template <typename T>
class optional {
  // ...
  template <typename Self>
  constexpr auto operator->(this Self&& self) {
    return addressof(self.m_value);
  }
  // ...
};
```

```cpp
template <typename T>
class optional {
  // ...
  constexpr T& operator*() & {
    return this->m_value;
  }

  constexpr T const& operator*() const& {
    return this->m_value;
  }

  constexpr T&& operator*() && {
    return move(this->m_value);
  }

  constexpr T const&&
  operator*() const&& {
    return move(this->m_value);
  }

  constexpr T& value() & {
    if (has_value()) {
      return this->m_value;
    }
    throw bad_optional_access();
  }

  constexpr T const& value() const& {
    if (has_value()) {
      return this->m_value;
    }
    throw bad_optional_access();
  }

  constexpr T&& value() && {
    if (has_value()) {
      return move(this->m_value);
    }
    throw bad_optional_access();
  }

  constexpr T const&& value() const&& {
    if (has_value()) {
      return move(this->m_value);
    }
    throw bad_optional_access();
  }
  // ...
};
```

```cpp
template <typename T>
class optional {
  // ...
  template <typename Self>
  constexpr like_t<Self, T>&& operator*(this Self&&
    return forward<Self>(self).m_value;
  }

  template <typename Self>
  constexpr like_t<Self, T>&& value(this Self&& sel
    if (this->has_value()) {
      return forward<Self>(self).m_value;
    }
    throw bad_optional_access();
  }
  // ...
};
```

```
template <typename T>
class optional {
  // ...
  template <typename F>
  constexpr auto and_then(F&& f) & {
    using result =
      invoke_result_t<F, T&>;
    static_assert(
      is_optional<result>::value,
      "F must return an optional");

    return has_value()
        ? invoke(forward<F>(f), **this)
        : nullopt;
  }

  template <typename F>
  constexpr auto and_then(F&& f) && {
    using result =
      invoke_result_t<F, T&&>;
    static_assert(
      is_optional<result>::value,
      "F must return an optional");

    return has_value()
        ? invoke(forward<F>(f),
                 move(**this))
        : nullopt;
  }

  template <typename F>
  constexpr auto and_then(F&& f) const& {
    using result =
      invoke_result_t<F, T const&>;
    static_assert(
      is_optional<result>::value,
      "F must return an optional");

    return has_value()
        ? invoke(forward<F>(f), **this)
        : nullopt;
  }

  template <typename F>
  constexpr auto and_then(F&& f) const&& {
    using result =
      invoke_result_t<F, T const&&>;
    static_assert(
      is_optional<result>::value,
      "F must return an optional");

    return has_value()
        ? invoke(forward<F>(f),
                 move(**this))
        : nullopt;
  }
  // ...
};
```

```
template <typename T>
class optional {
  // ...
  template <typename Self, typename F>
  constexpr auto and_then(this Self&& self, F&& f)
    using val = decltype((
        forward<Self>(self).m_value));
    using result = invoke_result_t<F, val>;

    static_assert(
      is_optional<result>::value,
      "F must return an optional");

    return this->has_value()
        ? invoke(forward<F>(f),
                 forward<Self>(self).m_value)
        : nullopt;
  }
  // ...
};
```

There are a few more functions in P0798 responsible for this explosion of overloads, so the difference in both code and clarity is dramatic.

For those that dislike returning auto in these cases, it is easy to write a metafunction matching the appropriate qualifiers from a type. It is certainly a better option than blindly copying and pasting code, hoping that the minor changes were made correctly in each case.

§ 4.2. CRTP, without the C, R, or even T

Today, a common design pattern is the Curiously Recurring Template Pattern. This implies passing the derived type as a template parameter to a base class template as a way of achieving static polymorphism. If we wanted to simply outsource implementing postfix incrementation to a base, we could use CRTP for that. But with explicit objects that already deduce to the derived objects, we don't need any curious recurrence — we can use standard inheritance and let deduction do its thing. The base class doesn't even need to be a template:

C++17

```
template <typename Derived>
struct add_postfix_increment {
    Derived operator++(int) {
        auto& self = static_cast<Derived&>(*this);

        Derived tmp(self);
        ++self;
        return tmp;
    }
};

struct some_type : add_postfix_increment<some_type> {
    some_type& operator++() { ... }
};
```

Proposed

```
struct add_postfix_increment {
    template <typename Self>
    auto operator++(this Self&& self, int) {
        auto tmp = self;
        ++self;
        return tmp;
    }
};



struct some_type : add_postfix_increment {
    some_type& operator++() { ... }
};
```

The proposed examples aren't much shorter, but they are certainly simpler by comparison.

§ 4.2.1. Builder pattern

Once we start to do any more with CRTP, complexity quickly increases, whereas with this proposal, it stays remarkably low.

Let's say we have a builder that does multiple things. We might start with:

```
struct Builder {
  Builder& a() { /* ... */; return *this; }
  Builder& b() { /* ... */; return *this; }
  Builder& c() { /* ... */; return *this; }
};

Builder().a().b().a().b().c();
```

But now we want to create a specialized builder with new operations `d()` and `e()`. This specialized builder needs new member functions, and we don't want to burden existing users with them. We also want `Special().a().d()` to work, so we need to use CRTP to *conditionally* return either a `Builder&` or a `Special&`:

C++17	Proposed

```cpp
template <typename D=void>
class Builder {
  using Derived = conditional_t<is_void_v<D>, Builder, D>;
  Derived& self() {
    return *static_cast<Derived*>(this);
  }

public:
  Derived& a() { /* ... */; return self(); }
  Derived& b() { /* ... */; return self(); }
  Derived& c() { /* ... */; return self(); }
};

struct Special : Builder<Special> {
  Special& d() { /* ... */; return *this; }
  Special& e() { /* ... */; return *this; }
};

Builder().a().b().a().b().c();
Special().a().d().e().a();
```

```cpp
struct Builder {
    template <typename Self>
    Self& a(this Self&& self) { /* ... */;

    template <typename Self>
    Self& b(this Self&& self) { /* ... */;

    template <typename Self>
    Self& c(this Self&& self) { /* ... */;
};

struct Special : Builder {
    Special& d() { /* ... */; return *this
    Special& e() { /* ... */; return *this
};

Builder().a().b().a().b().c();
Special().a().d().e().a();
```

The code on the right is dramatically easier to understand and therefore more accessible to more programmers than the code on the left.

But wait! There's more!

What if we added a *super*-specialized builder, a more special form of `Special`? Now we need `Special` to opt-in to CRTP so that it knows which type to pass to `Builder`, ensuring that everything in the hierarchy returns the correct type. It's about this point that most programmers would give up. But with this proposal, there's no problem!

C++17	Proposed

```cpp
template <typename D=void>
class Builder {
protected:
  using Derived = conditional_t<is_void_v<D>, Builder, D>;
  Derived& self() {
    return *static_cast<Derived*>(this);
  }

public:
  Derived& a() { /* ... */; return self(); }
  Derived& b() { /* ... */; return self(); }
  Derived& c() { /* ... */; return self(); }
};

template <typename D=void>
struct Special
  : Builder<conditional_t<is_void_v<D>,Special<D>,D>
{
  using Derived = typename Special::Builder::Derived;
  Derived& d() { /* ... */; return this->self(); }
  Derived& e() { /* ... */; return this->self(); }
};

struct Super : Special<Super>
{
    Super& f() { /* ... */; return *this; }
};

Builder().a().b().a().b().c();
Special().a().d().e().a();
Super().a().d().f().e();
```

```cpp
struct Builder {
    template <typename Self>
    Self& a(this Self&& self) { /* ... */;

    template <typename Self>
    Self& b(this Self&& self) { /* ... */;

    template <typename Self>
    Self& c(this Self&& self) { /* ... */;
};

struct Special : Builder {
    template <typename Self>
    Self& d(this Self&& self) { /* ... */;

    template <typename Self>
    Self& e(this Self&& self) { /* ... */;
};

struct Super : Special {
    template <typename Self>
    Self& f(this Self&& self) { /* ... */;
};

Builder().a().b().a().b().c();
Special().a().d().e().a();
Super().a().d().f().e();
```

The code on the right is much easier in all contexts. There are so many situations where this idiom, if available, would give programmers a better solution for problems that they cannot easily solve today.

Note that the `Super` implementations with this proposal opt-in to further derivation, since it's a no-brainer at this point.

§ 4.3. Recursive Lambdas

The explicit object parameter syntax offers an alternative solution to implementing a recursive lambda as compared to [P0839R0], since now we've opened up the possibility of allowing a lambda to reference itself. To do this, we need a way to *name* the lambda.

```cpp
// as proposed in P0839
auto fib = [] self (int n) {
    if (n < 2) return n;
    return self(n-1) + self(n-2);
};

// this proposal
auto fib = [](this auto const& self, int n) {
    if (n < 2) return n;
    return self(n-1) + self(n-2);
};
```

This works by following the established rules. The call operator of the closure object can also have an explicit object parameter, so in this example, `self` is the closure object.

There was some concern in San Diego about the implementability of this aspect of the proposal. But any issues would come from having a non-dependent way to identify the lambda object itself - any such uses, even `sizeof()`, would be problematic because the lambda is not yet complete. But because the `self` parameter of the call operator is deduced, and

that is the only way this proposal is offering to access the lambda itself, there are no problems. The lambda will be complete by instantiation time, so everything works.

Combine this with the new style of mixins allowing us to automatically deduce the most derived object, and you get the following example — a simple recursive lambda that counts the number of leaves in a tree.

```
struct Node;
using Tree = variant<Leaf, Node*>;
struct Node {
    Tree left;
    Tree right;
};

int num_leaves(Tree const& tree) {
    return visit(overload(        // <-----------------------------------+
        [](Leaf const&) { return 1; },                                  //      |
        [](this auto const& self, Node* n) -> int {                     //      |
            return visit(self, n->left) + visit(self, n->right); // <----+
        }
    ), tree);
}
```

In the calls to `visit`, `self` isn't the lambda; `self` is the `overload` wrapper. This works straight out of the box.

4.4. By-value member functions

This section presents some of the cases for by-value member functions.

4.4.1. For move-into-parameter chaining

Say you wanted to provide a `.sorted()` method on a data structure. Such a method naturally wants to operate on a copy. Taking the parameter by value will cleanly and correctly move into the parameter if the original object is an rvalue without requiring templates.

```
struct my_vector : vector<int> {
  auto sorted(this my_vector self) -> my_vector {
    sort(self.begin(), self.end());
    return self;
  }
};
```

4.4.2. For performance

It's been established that if you want the best performance, you should pass small types by value to avoid an indirection penalty. One such small type is `std::string_view`. Abseil Tip #1 for instance, states:

> Unlike other string types, you should pass `string_view` by value just like you would an `int` or a `double` because `string_view` is a small value.

There is, however, one place today where you simply *cannot* pass types like `string_view` by value: to their own member functions. The implicit object parameter is always a reference, so any such member functions that do not get inlined incur a double indirection.

As an easy performance optimization, any member function of small types that does not perform any modifications can take the object parameter by value. Here is an example of some member functions of `basic_string_view` assuming that we are just using `charT const*` as `iterator`:

```
template <class charT, class traits = char_traits<charT>>
class basic_string_view {
private:
    const_pointer data_;
    size_type size_;
public:
    constexpr const_iterator begin(this basic_string_view self) {
        return self.data_;
    }

    constexpr const_iterator end(this basic_string_view self) {
        return self.data_ + self.size_;
    }

    constexpr size_t size(this basic_string_view self) {
        return self.size_;
    }

    constexpr const_reference operator[](this basic_string_view self, size_type pos) {
        return self.data_[pos];
    }
};
```

Most of the member functions can be rewritten this way for a free performance boost.

The same can be said for types that aren't only cheap to copy, but have no state at all. Compare these two implementations of `less_than`:

C++17	Proposed

`struct less_than {`
`  template <typename T, typename U>`
`  bool operator()(T const& lhs, U const& rhs) {`
`  return lhs < rhs;`
`  }`
`};`
	`struct less_than {`
	`  template <typename T, typename U>`
	`  bool operator()(this less_than,`
	`  T const& lhs, U const& rhs) {`
	`  return lhs < rhs;`
	`  }`
	`};`

In C++17, invoking `less_than()(x, y)` still requires an implicit reference to the `less_than` object — completely unnecessary work when copying it is free. The compiler knows it doesn't have to do anything. We *want* to pass `less_than` by value here. Indeed, this specific situation is the main motivation for [P1169R0].

§ 4.5. SFINAE-friendly callables

A seemingly unrelated problem to the question of code quadruplication is that of writing numerous overloads for function wrappers, as demonstrated in [P0826R0]. Consider what happens if we implement `std::not_fn()` as currently specified:

```
template <typename F>
class call_wrapper {
    F f;
public:
    // ...
    template <typename... Args>
    auto operator()(Args&&... ) &
        -> decltype(!declval<invoke_result_t<F&, Args...>>());

    template <typename... Args>
    auto operator()(Args&&... ) const&
        -> decltype(!declval<invoke_result_t<F const&, Args...>>());

    // ... same for && and const && ...
};

template <typename F>
auto not_fn(F&& f) {
    return call_wrapper<decay_t<F>>{forward<F>(f)};
}
```

As described in the paper, this implementation has two pathological cases: one in which the callable is SFINAE-unfriendly, causing the call to be ill-formed where it would otherwise work; and one in which overload is deleted, causing the call to fall back to a different overload when it should fail instead:

```
struct unfriendly {
    template <typename T>
    auto operator()(T v) {
        static_assert(is_same_v<T, int>);
        return v;
    }

    template <typename T>
    auto operator()(T v) const {
        static_assert(is_same_v<T, double>);
        return v;
    }
};

struct fun {
    template <typename... Args>
    void operator()(Args&&...) = delete;

    template <typename... Args>
    bool operator()(Args&&...) const { return true; }
};

std::not_fn(unfriendly{})(1); // static assert!
                              // even though the non-const overload is viable and would be
                              // best match, during overload resolution, both overloads of
                              // unfriendly have to be instantiated - and the second one is
                              // hard compile error.

std::not_fn(fun{})();         // ok!? Returns false
                              // even though we want the non-const overload to be deleted,
                              // const overload of the call_wrapper ends up being viable -
                              // the only viable candidate.
```

Gracefully handling SFINAE-unfriendly callables is **not solvable** in C++ today. Preventing fallback can be solved by the addition of another four overloads, so that each of the four *cv*/ref-qualifiers leads to a pair of overloads: one enabled and one `deleted`.

This proposal solves both problems by allowing `this` to be deduced. The following is a complete implementation of `std::not_fn`. For simplicity, it makes use of `BOOST_HOF_RETURNS` from Boost.HOF to avoid duplicating expressions:

```
template <typename F>
struct call_wrapper {
  F f;

  template <typename Self, typename... Args>
  auto operator()(this Self&& self, Args&&... args)
    BOOST_HOF_RETURNS(
      !invoke(
        forward<Self>(self).f,
        forward<Args>(args)...))
};

template <typename F>
auto not_fn(F&& f) {
  return call_wrapper<decay_t<F>>{forward<F>(f)};
}
```

Which leads to:

```
not_fn(unfriendly{})(1); // ok
not_fn(fun{})();         // error
```

Here, there is only one overload with everything deduced together. The first example now works correctly. `Self` gets deduced as `call_wrapper<unfriendly>`, and the one `operator()` will only consider `unfriendly`'s non-`const` call operator. The `const` one is never even considered, so it does not have an opportunity to cause problems.

The second example now also fails correctly. Previously, we had four candidates. The two non-`const` options were removed from the overload set due to `fun`'s non-`const` call operator being `delete`d, and the two `const` ones which were viable. But now, we only have one candidate. `Self` is deduced as `call_wrapper<fun>`, which requires `fun`'s non-`const` call operator to be well-formed. Since it is not, the call results in an error. There is no opportunity for fallback since only one overload is ever considered.

This singular overload has precisely the desired behavior: working for `unfriendly`, and not working for `fun`.

This could also be implemented as a lambda completely within the body of `not_fn`:

```
template <typename F>
auto not_fn(F&& f) {
    return [f=forward<F>(f)](this auto&& self, auto&&.. args)
        BOOST_HOF_RETURNS(
            !invoke(
                forward_like<decltype(self)>(f),
                forward<decltype(args)>(args)...))
        ;
}
```

§ 5. Acknowledgements

The authors would like to thank:

- Jonathan Wakely, for bringing us all together by pointing out we were writing the same paper, twice
- Chandler Carruth for a lot of feedback and guidance around many design issues, but especially for help with use cases and the pointer-types for by-value passing
- Graham Heynes, Andrew Bennieston, Jeff Snyder for early feedback regarding the meaning of `this` inside function bodies

- Amy Worthington, Jackie Chen, Vittorio Romeo, Tristan Brindle, Agustín Bergé, Louis Dionne, and Michael Park for early feedback
- Guilherme Hartmann for his guidance with the implementation
- Richard Smith, Jens Maurer, and Hubert Tong for help with wording
- Ville Voutilainen, Herb Sutter, Titus Winters and Bjarne Stroustrup for their guidance in design-space exploration
- Eva Conti for furious copy editing, patience, and moral support

§ References

§ Informative References

[Effective]
Scott Meyers. Effective C++, Third Edition. 2005. URL: https://www.aristeia.com/books.html

[P0798R0]
Simon Brand. Monadic operations for std::optional. 6 October 2017. URL: https://wg21.link/p0798r0

[P0826R0]
Agustín Bergé. SFINAE-friendly std::bind. 12 October 2017. URL: https://wg21.link/p0826r0

[P0839R0]
Richard Smith. Recursive Lambdas. 16 October 2017. URL: https://wg21.link/p0839r0

[P0847R0]
Gašper Ažman, Simon Brand, Ben Deane, Barry Revzin. Deducing this. 12 February 2018. URL: https://wg21.link/p0847r0

[P0847R1]
Gašper Ažman, Simon Brand, Ben Deane, Barry Revzin. Deducing this. 7 October 2018. URL: https://wg21.link/p0847r1

[P0929R2]
Jens Maurer. Checking for abstract class types. 6 June 2018. URL: https://wg21.link/p0929r2

[P1169R0]
Barry Revzin. static operator(). 7 October 2018. URL: https://wg21.link/p1169r0